NILEBRIDGE PROFESSIONAL ACADEMY

Applied Artificial Intelligence & Machine Learning

DEMONSTRATION CATALOGUE · 2026–2027 · CATALOGUE CODE AI

CATALOGUE OVERVIEW

A structured learning portfolio covering Python foundations, applied machine learning, and document-grounded LLM applications. Each programme includes entry requirements, outcomes, a detailed curriculum, practical labs, assessment criteria, project specifications, and learner support notes.

ACADEMY AND DATA NOTICE

NILEBRIDGE PROFESSIONAL ACADEMY is a fictional demonstration institution. All programme records, durations, curriculum details, assessments, and career descriptions are synthetic examples created for RAG ingestion and retrieval testing. They are not real academy offerings, verified schedules, official accreditation claims, or employment guarantees.

Programme Directory

The programme records below use stable programme identifiers to support retrieval and metadata validation.

Programme ID	Programme title	Level	Duration
AI-FND-101	Python & Data Foundations	Foundation	60 guided learning hours
AI-ML-201	Applied Machine Learning	Intermediate	100 guided learning hours
AI-RAG-301	LLM Applications & Retrieval-Augmented Generation	Advanced foundation	120 guided learning hours

Information included for each programme

- Programme identity, level, duration, delivery format, and intended audience
- Entry requirements and programme purpose
- Learning outcomes and week-by-week curriculum
- Practical labs, assessment weights, and grading criteria
- Capstone brief, deliverables, tools, and FAQ
- Provenance and accreditation status

PROGRAMME 01 · AI-FND-101

Python & Data Foundations

Indicative hours	Duration	Level	Delivery
60 guided learning hours	8 weeks	Foundation	Instructor-led workshops with guided labs and independent exercises.

Programme summary

Introduces Python programming and structured-data handling. Learners build confidence writing small programs, debugging common errors, and producing reproducible notebook-based work.

Intended audience

Learners preparing for AI, analytics, or software-development study.

Entry requirements

Basic computer literacy. No prior programming is required.

Learning outcomes

- Write functions using variables, conditions, loops, and collections.
- Read and write CSV and JSON files and validate basic data assumptions.
- Use NumPy and pandas for filtering, transformation, grouping, and summary analysis.
- Create clear visualizations and document findings in a reproducible notebook.
- Organize a small project with dependencies, a README, and version control.

Week-by-week curriculum

Week	Topic	Coverage	Practice / evidence
1	Python environment and first programs	Install/check Python, use an editor or notebook, print values, work with variables and basic types.	Create a notebook with simple input/output examples.
2	Conditions, loops, and functions	Write conditional logic, repeat operations, define functions, and decompose a task.	Build a reusable set of small calculation functions.
3	Collections and error handling	Use lists and dictionaries, iterate over records, handle expected errors, and validate input.	Process a list of structured learner or product records.
4	Files and project structure	Read and write CSV/JSON, separate code into modules, use virtual environments, and record dependencies.	Create a small command-line data utility.
5	NumPy and pandas	Create arrays and DataFrames, select columns, filter rows, group data, and join tables.	Analyze a tabular dataset and explain each transformation.
6	Data quality and exploration	Inspect missing values, duplicates, data types, distributions, and suspicious records.	Produce a data-quality checklist and a cleaned dataset.
7	Visualization and communication	Select charts, label units, explain trends, and distinguish observations from assumptions.	Create three charts and a short findings note.
8	Capstone and review	Combine ingestion, cleaning, analysis, documentation, and a reproducible run-through.	Present the final project and answer reviewer questions.

Practical labs

- Python syntax and debugging exercises
- CSV/JSON parsing and validation
- pandas cleaning and grouping
- Visualization critique and revision

Assessment structure

Component	Weight	Evidence / criteria
Weekly practice	20%	Correctness, readable code, and completion of guided exercises.
Data notebook	25%	Appropriate transformations, validation checks, and clear explanations.

Component	Weight	Evidence / criteria
Practical quiz	15%	Ability to interpret code and repair small errors.
Capstone project	40%	Working solution, reproducibility, documentation, and presentation.

Capstone project

Build a small analysis pipeline that reads a supplied CSV, checks required columns, reports missing and duplicate records, produces summary statistics, and saves a notebook with charts and conclusions.

Suggested grading rubric

- Correctness and data validation: 30%
- Code organization and readability: 20%
- Analysis and visual communication: 25%
- Reproducibility and documentation: 25%

Tools and technologies

Python 3 · Jupyter Notebook · NumPy · pandas · Matplotlib · Git

Frequently asked questions

Do I need previous coding experience?

No. Basic computer literacy is expected; the programme starts with core Python concepts.

Does the course guarantee a job?

No. It provides learning activities and portfolio evidence; hiring depends on broader skills and circumstances.

What should I submit for the capstone?

A working notebook or script, a sample input dataset, a README, and a short findings summary.

Record and provenance

Department: Artificial Intelligence | Skill level: Foundation | Accreditation: not verified | Country: not specified | Source status: synthetic demonstration data | Catalogue: Applied Artificial Intelligence & Machine Learning | Programme: Python & Data Foundations | Source file: 01_AI_Machine_Learning_Expanded_Catalogue.pdf.

PROGRAMME 02 · AI-ML-201

Applied Machine Learning

Indicative hours	Duration	Level	Delivery
100 guided learning hours	10 weeks	Intermediate	Instructor-led technical sessions, labs, code review, and project mentoring.

Programme summary

Covers the end-to-end supervised machine-learning workflow: problem definition, baseline creation, data preparation, training, validation, metric selection, error analysis, and communication of limitations.

Intended audience

Learners with Python experience who want to build and evaluate practical ML models.

Entry requirements

Python functions and collections, basic descriptive statistics, and comfort with tabular data.

Learning outcomes

- Frame a business or research question as a measurable ML task.
- Build preprocessing and model pipelines without leaking information from validation data.
- Train and compare regression and classification models.
- Select evaluation metrics appropriate to class balance and error costs.
- Interpret model errors and document data limitations and risks.

Week-by-week curriculum

Week	Topic	Coverage	Practice / evidence
1	Problem framing and baselines	Define target, prediction unit, success metric, baseline, and data assumptions.	Write an ML problem statement and baseline plan.
2	Splitting and leakage	Create train/validation/test splits and identify leakage from preprocessing or duplicated records.	Audit a sample workflow for leakage risks.
3	Preprocessing pipelines	Handle missing values, scale numerical data, encode categories, and combine transformations.	Build a reusable preprocessing pipeline.
4	Regression	Train linear and tree-based regressors and compare error measures.	Submit a regression comparison notebook.
5	Classification	Train logistic regression and tree-based classifiers; interpret class probabilities.	Evaluate a classifier with a confusion matrix.
6	Model selection	Use cross-validation and a limited hyperparameter search with a fixed evaluation protocol.	Submit a model-selection record.
7	Imbalanced data and thresholds	Compare precision, recall, F1, and threshold trade-offs for different error costs.	Recommend a threshold with explicit rationale.
8	Unsupervised learning overview	Explore clustering and dimensionality reduction, including limits of interpretation.	Produce a cluster exploration note.
9	Interpretation and error analysis	Review error slices, feature influence, fairness considerations, and generalization risks.	Write a limitations and error-analysis report.
10	Capstone review	Package the pipeline, evaluate on held-out data, and communicate results.	Present a reproducible model and evaluation report.

Practical labs

- Regression model comparison
- Classification and confusion-matrix analysis
- Leakage audit and pipeline repair
- Cross-validation and error-slice review

Assessment structure

Component	Weight	Evidence / criteria
Technical labs	25%	Reproducible experiments and appropriate use of training/validation data.
Model review	20%	Metric choice, comparison fairness, and interpretation of results.
Practical assessment	15%	Diagnose a flawed workflow and explain the correction.
Capstone	40%	Complete pipeline, evidence-based evaluation, and limitations.

Capstone project

Choose an approved tabular dataset, establish a baseline, compare at least two candidate models, use an appropriate validation strategy, report task-relevant metrics, inspect errors, and document limitations.

Suggested grading rubric

- Problem framing and baseline: 15%
- Data handling and leakage prevention: 25%
- Model evaluation and error analysis: 30%
- Reproducibility and communication: 30%

Tools and technologies

Python · pandas · scikit-learn · Matplotlib · Jupyter · Git

Frequently asked questions

Is deep learning required?

No. This programme focuses on practical classical ML workflows; deep learning is covered separately.

Can the highest accuracy model automatically win?

No. The model should be chosen using suitable metrics, validation design, error costs, and generalization evidence.

What is data leakage?

Leakage occurs when information unavailable at prediction time improperly influences training or evaluation, producing misleading results.

Record and provenance

Department: Artificial Intelligence | Skill level: Intermediate | Accreditation: not verified | Country: not specified | Source status: synthetic demonstration data | Catalogue: Applied Artificial Intelligence & Machine Learning | Programme: Applied Machine Learning | Source file: 01_AI_Machine_Learning_Expanded_Catalogue.pdf.

PROGRAMME 03 · AI-RAG-301

LLM Applications & Retrieval-Augmented Generation

Indicative hours	Duration	Level	Delivery
120 guided learning hours	12 weeks	Advanced foundation	Technical workshops, architecture reviews, guided implementation, and evaluation sessions.

Programme summary

Teaches a retrieval-augmented generation workflow that parses source documents, preserves provenance, creates coherent chunks, indexes embeddings, retrieves relevant passages, and generates evidence-grounded answers.

Intended audience

Developers and AI learners building document-grounded applications.

Entry requirements

Python, JSON, basic APIs, and familiarity with testing; introductory ML knowledge is recommended.

Learning outcomes

- Design a modular ingestion pipeline for supported file formats.
- Preserve source file, page, section, and chunk identifiers throughout processing.
- Compare chunking and embedding choices using representative questions.
- Implement retrieval and answer generation with source references.
- Test answer grounding and handle cases where evidence is missing or conflicting.

Week-by-week curriculum

Week	Topic	Coverage	Practice / evidence
1	RAG architecture and test design	Separate ingestion, indexing, retrieval, generation, and evaluation responsibilities.	Draw a system diagram and define initial evaluation questions.
2	File discovery and parsing	Identify file formats and extract text, tables, slides, and page context where supported.	Parse a small mixed-format document set.
3	Cleaning and normalization	Normalize whitespace, detect repeated page furniture conservatively, and record parser limitations.	Compare original and cleaned extraction outputs.
4	Chunking and metadata	Use document-aware boundaries, semantic grouping, and recursive splitting for oversized sections.	Inspect chunks for topic coherence and source metadata.
5	Embeddings and vector indexing	Generate embeddings, store vectors with metadata, and define index lifecycle operations.	Build and query a small vector index.
6	Retrieval quality	Tune top-k and filters, inspect relevant/irrelevant passages, and create retrieval test cases.	Produce a retrieval comparison report.
7	Grounded generation	Pass retrieved context with source labels and instruct the model not to invent missing facts.	Demonstrate sourced answers and evidence-insufficient responses.
8	Arabic and English queries	Test language detection, dedicated translation options, mixed-language input, and original-language responses.	Build a multilingual query test matrix.
9	Service boundaries and MCP	Expose narrowly scoped search tools and validate input/output contracts for agent use.	Draft an MCP tool interface and test its contract.
10	Evaluation and failure analysis	Measure retrieval relevance, answer support, citation correctness, and refusal behavior.	Label and analyze a representative test set.
11	Reliability and observability	Handle corrupt files, empty extraction, duplicate ingestion, timeouts, and logging.	Create ingestion error tests and a troubleshooting guide.
12	Capstone and demo	Run the pipeline end to end and demonstrate both successful and unsupported questions.	Submit a working prototype and evaluation report.

Practical labs

- Mixed-format parsing
- Chunk inspection and metadata validation

- Retrieval top-k and filter experiments
- Grounded answer and citation checks
- MCP interface contract test

Assessment structure

Component	Weight	Evidence / criteria
Ingestion implementation	20%	File support, error handling, and source preservation.
Retrieval experiment	20%	Question set, comparison method, and interpretation.
Grounded-answer behavior	25%	Evidence support, citation correctness, and uncertainty handling.
Capstone and documentation	35%	Working pipeline, tests, runbook, and demonstration.

Capstone project

Build a RAG prototype for a controlled collection of course documents. It must ingest files, save inspectable chunk records, index embeddings, retrieve source passages, answer in the user's language where supported, and provide source references.

Suggested grading rubric

- Pipeline modularity and reliability: 20%
- Retrieval relevance: 25%
- Grounded answer and citation behavior: 25%
- Testing, documentation, and reproducibility: 30%

Tools and technologies

Python · parsers for selected formats · embedding model · vector store such as Chroma · local or hosted LLM · pytest · optional FastAPI/MCP SDK

Frequently asked questions

Does RAG guarantee that every answer is correct?

No. Retrieval and generation can fail; the system must be evaluated and should acknowledge insufficient evidence.

Should the same embedding model be used for documents and queries?

Normally yes, unless the chosen retrieval design explicitly supports a different compatible method.

Does a vector database replace the source files?

No. Keep original documents and traceable metadata; the index is a retrieval representation.

Does completing this course certify a system for production?

No. Production readiness requires separate security, privacy, reliability, and operational review.

Record and provenance

Department: Artificial Intelligence | Skill level: Advanced foundation | Accreditation: not verified | Country: not specified | Source status: synthetic demonstration data | Catalogue: Applied Artificial Intelligence & Machine Learning | Programme: LLM Applications & Retrieval-Augmented Generation | Source file: 01_AI_Machine_Learning_Expanded_Catalogue.pdf.